

UPSCALING ENGINE FOR MODDERS

PixelForge Engine

Old game textures in.
4x HD out.

Seamless tiles

Keeps alpha

Batch whole packs



THE PROBLEM

HD screens expose every old texture.

1

Remasters need thousands of textures

Redrawing every wall, floor and prop by hand is the slowest, most expensive part of a remaster or HD mod.

2

Simple resizing just blurs

Stretching a texture 4× with bicubic gives soft, muddy surfaces that look worse on a modern monitor.

3

Generic AI upscalers aren't tuned for games

Tools trained on photos can smear tiling patterns or invent the wrong kind of detail.

PixelForge does the first pass for you: **seconds per texture, in your browser.**

TRY IT YOURSELF

Drop in a texture. Drag the slider.

AI FOR GAME ARTISTS AND MODDERS

PixelForge

Old game textures in . 4x HD out.

4x upscaling

Trained on game textures

Runs in your browser

1 - Pick a low-res texture



Slumped Mortar Brick



Aerial Rocks 02



Bark Blue gum



Brick Pavement 03



Brushed Concrete 2



Clay Floor 001

...or upload your own

Works best on textures, tiles and painted art.
Pixel-art sprites need a different model. Large images are processed in tiles. 🚀 Your image never leaves your device.

vs Bicubic

vs Original pixels

vs True HD



PixelForge



128x128 → 512x512 in 2.3 s, in your browser

Download HD PNG

1

Before/after slider

Compare against bicubic, the original pixels, or the true HD texture.

2

Runs in your browser

No upload, no server, no install. Your art never leaves your machine.

3

Batch + ZIP download

Drop in many textures at once and get one ZIP back (2.3 s per 512x512 result in a laptop browser).

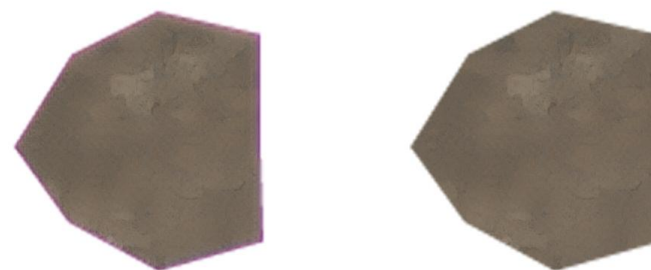
BUILT FOR MODDERS

An engine you can drop into your mod pipeline.

Feature	Plain upscaling	PixelForge Engine
Seamless tiles	24 of 30 textures show a seam	2 of 30
Transparent edges	colour halo (edge error 16.1)	1.6 (-90%)
Whole texture packs	one file at a time	0.20 s / texture GPU · 0.55 s CPU



Tiling seam: normal (left) vs --tileable (right)



Transparent decal: halo (left) vs clean edge (right)

Command line

```
pip install git+https://github.com/Priyanshu-Singh-git/pixelforge
pixelforge upscale ./textures ./textures_hd
pixelforge upscale ./tiles ./tiles_hd --tileable
pixelforge export PixelForge_4x.pth
```

Folder in, folder out · names and sub-folders kept · GPU or CPU · ONNX + ESRGAN-format weights

RESULTS

Looks like real HD. Small enough for a browser.

41%

closer to the real HD look than bicubic

LPIPS perceptual distance, 60 unseen game textures

15×

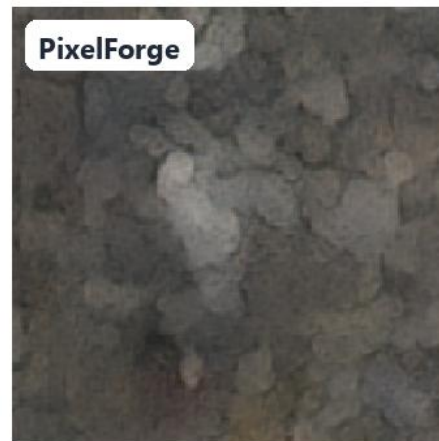
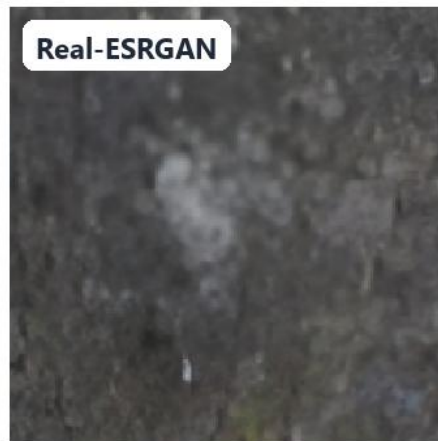
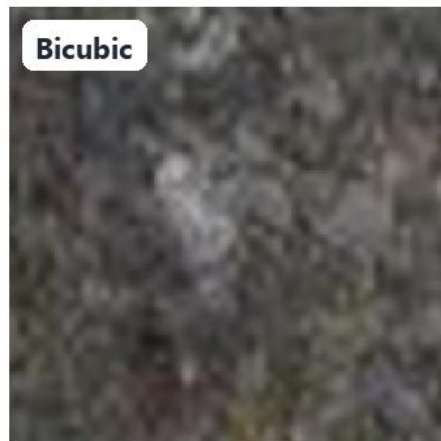
smaller than Real-ESRGAN (1.1M vs 16.7M parameters)

LPIPS 0.414 vs 0.454 (lower is better)

2.3 s

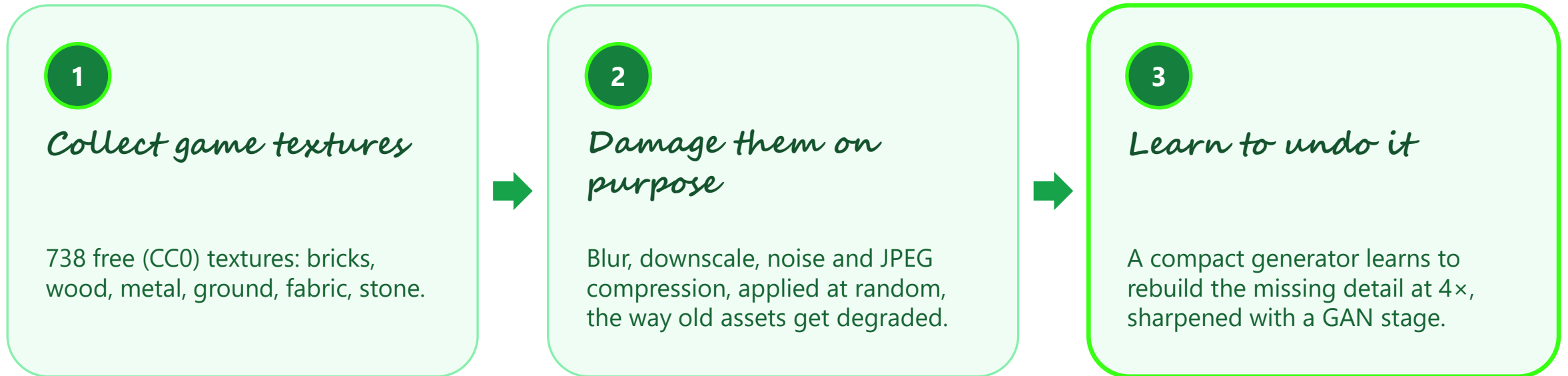
to upscale a texture 4× in a laptop browser

128×128 → 512×512, no GPU, no upload



HOW IT WORKS

Trained to undo old-texture damage.

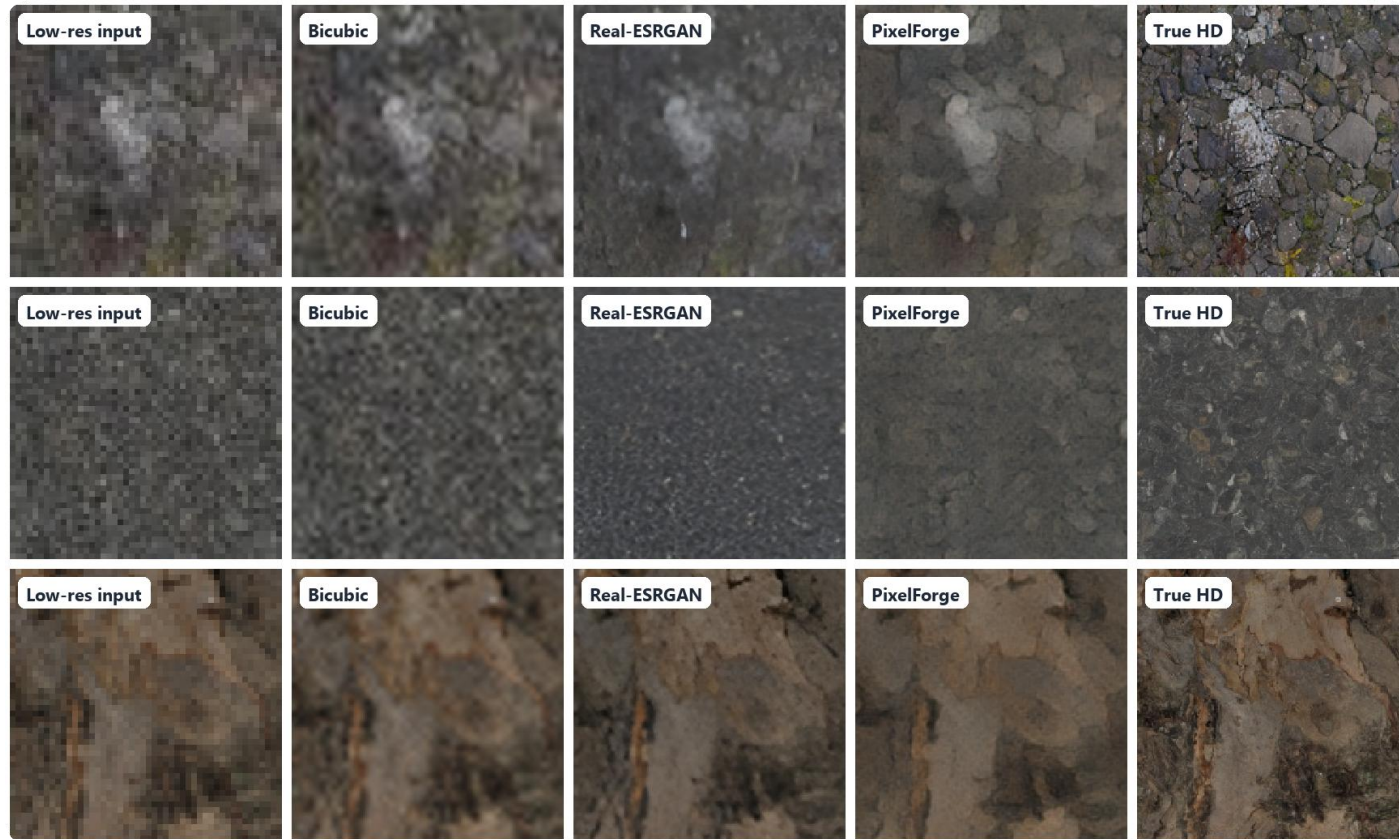


For engineers: compact ESRGAN-style RRDB generator (1.1M params) trained from scratch in PyTorch: L1 stage, then VGG-perceptual + U-Net-discriminator GAN stage with EMA (Real-ESRGAN recipe), exported to ONNX. Benchmarks are in the appendix.

REAL RESULTS

Real results on textures it never saw.

Median-improvement textures, different materials, zoomed. PixelForge clears noise and JPEG blocks; detail the tiny input no longer has can't be fully recovered.



HOW WE'D WORK TOGETHER

Start with a test pack. Scale to the whole library.

STEP 1

Send 10-20 textures

A representative sample of your game's textures or asset pack.

STEP 2

Test pack back

I upscale them, optionally fine-tune on your art style, and send before/after comparisons.

STEP 3

Batch the library

Folder in, folder out, file names kept, ready to drop into your mod or engine.

What you get: the HD texture set · a batch tool you can rerun on new assets · optional model fine-tuned on your art style · before/after report

NEXT STEP

*Use the engine on your mod today,
or send me 10 textures to try.*



Try the live demo

<https://priyanshu-singh-git.github.io/pixelforge/>

Engine code, CLI and benchmark

<https://github.com/Priyanshu-Singh-git/pixelforge>

The numbers behind it.

Held-out test set: 60 game textures, realistic degradation, ×4. PSNR/SSIM higher is better; LPIPS lower is better.

Method	Params	PSNR ↑	SSIM ↑	LPIPS ↓	GPU ms
Bicubic	–	29.31	0.612	0.707	–
Lanczos	–	29.24	0.608	0.711	–
Real-ESRGAN x4plus (pretrained)	16.70M	27.76	0.545	0.454	185.4
Real-ESRGAN general-v3 (pretrained)	1.21M	28.93	0.597	0.547	26.9
PixelForge SRVGG-S, L1 ★	0.16M	29.66	0.617	0.636	15.2
PixelForge RRDB-S, L1 ★	1.12M	29.73	0.620	0.659	39.0
PixelForge RRDB-S, GAN ★	1.12M	28.76	0.571	0.414	36.0

- Architecture: the 7× smaller SRVGG-S (0.16M) gets within 0.07 dB PSNR of RRDB-S (1.12M) and slightly better LPIPS (0.636 vs 0.659), at 15 vs 39 ms: a strong candidate for real-time use.
- L1 vs GAN: the GAN stage cuts LPIPS from 0.659 to 0.414 (looks far more real) at a cost of 0.97 dB PSNR, the classic fidelity vs realism trade-off.
- Versus Real-ESRGAN x4plus (trained on photos): LPIPS 0.414 vs 0.454, PSNR 28.76 vs 27.76 dB, with 15× fewer parameters.
- Single training run per model; synthetic degradations; pixel-art sprites and normal maps not yet handled.